

Smart Nutrition Assistant Companion & Curator (S.N.A.C.C.) ~ The AI Nutrition Assistant

Contributors: Faith Akinlade, Jared Simonetti, Kalyani Chaudhary, Ian Channer

Table of Contents

A. Problem Statement and Proposed Solution.....	2
B. Data Preprocessing and Cleaning Pipeline.....	3
C. Recommendation Engine: Model Pipeline and Filtering.....	6
D. Architectural Diagrams(v1 and v2).....	8
E. Challenges and Solutions.....	9
F. Project Contribution Summary.....	10
G. Sources.....	11

Github repository link:

<https://github.com/FaithAkinlade/AI-Nutrition-Assistant>

Note:

All instructions to run and files are located in the github repository.

A. Problem Statement

There is a significant problem of food waste in the US that affects people, the environment, and the economy. Food goes wasted in many different ways, whether it is retail, recalls, or food service. In America in particular, “around 35% of food produced in America goes unsold or uneaten. According to ReFed, if we recovered about half (46 billion pounds) of the food being wasted, we could feed every hungry person in the United States 3 meals a day, every day.” (moveforhunger, n.d.). This 35% of wasted food has a significant impact on both society at large and on individuals, economically and environmentally.

The environmental resources used for food products are often sustainable, including renewable resources like plants and sustainably farmed livestock. With this said, a consequence of the simple production of food in the modern world has a significant cost on the environment. This cost comes from the packaging and transportation of food products, methane production of livestock, elimination of forests in place of farmland, and management of food waste. These all contribute to increased greenhouse gas emissions and the consumption of natural resources (Nations, n.d.). The expense paid by the production of food becomes even more apparent when taking into account the scale of natural resources used on food production in the US. The precise statistics for this is as follows:

Half of the U.S. land, 80% of fresh water, and 10% of the energy budget is spent bringing food to our tables. When we waste 40% of food, we waste 40% of these natural resources. Worst yet, after wasted food ends up in landfills, it contributes to methane emissions that are 30 times worse than CO₂. We have an opportunity three times a day to reduce and eliminate this massive carbon footprint. - rescuingleftovercuisine.org

These statistics demonstrate the sheer magnitude of resources spent on food in America. Food production dominates the usage of land and consumption of water. Additionally, it is a non-insignificant contributor to the consumption of energy. These are resources we are overexerting in America as a consequence of food waste.

All of these statistics have a direct relationship with the wastage of economic resources. According to epa.gov, there is a clear impact that food waste has economically. Specifically, in 2023, the annual cost of food waste was \$728.14 per capita in the US. For a household of four, this number comes out to an average annual cost per household of \$2,912.56. Supplemented by census data from 2023 by census.gov (n.d.), the median household income was \$80,610. This additional statistic demonstrates that approximately ~3.61% of a household's income is being spent on food that is wasted. Additionally, in accordance with the population of America in 2023 from census.gov, there would be 334,233,854 individuals subject to food waste. This comes out to a total annual economic impact of \$243,369,038,451.56, approximately 250 billion dollars.

Food waste is the problem that the machine learning algorithm detailed in this paper addresses. The specific way that this machine learning solution addresses the problem is by allowing individuals to use ingredients and food items they already own to find recipes utilizing

those ingredients. This helps individuals and households make the most out of what they have and minimize the amount of food that ends up expiring or spoiling. This solution addresses food waste produced by households, which is part of the 19% of food waste which is sourced from retail, food service, and households according to moveforhunger.org (n.d.). This algorithm helps to address food waste in the world by targeting the household level of food waste.

B. Data Preprocessing and Cleaning Pipeline

The preprocessing layer serves as the foundation of SNACC. Its primary objective is to transform raw, unstructured recipe data into a standardized format suitable for vectorization and similarity analysis. This ensures that the recommendation engine treats "Freshly ground black pepper" and "pepper" as semantically related entities.

1. Raw Data Extraction The pipeline begins by ingesting the raw dataset (CSV format), and the initial steps involve:

- **Column Selection:** Retaining only the critical features: *Recipe_title*, *Ingredients*, *Directions*, and *Dietary_Profile*.
- **Missing Value Handling:** Identifying and removing entries with null values in the ingredient or instruction fields to prevent errors during tokenization.

2. Text Normalization To ensure consistency across the dataset, several string-level transformations are applied:

- **Lowercasing:** All text is converted to lowercase to ensure the model is case-insensitive.
- **Noise Removal:** Using Regular Expressions (Regex) to strip out special characters, punctuation, and non-alphabetic symbols.
- **Stop Word Filtering:** Common English words and redundant cooking measurements (e.g., "tbsp", "cups") are filtered out, allowing the TF-IDF vectorizer to prioritize unique ingredients.

3. Tokenization and Lemmatization Once the text is normalized, it is broken down into individual tokens (words).

- **Lemmatization:** We apply lemmatization to reduce words to their dictionary root (e.g., "tomatoes" to "tomato"). This preserves the linguistic accuracy of ingredient matching compared to basic stemming.

4. Feature Synthesis To provide a holistic view of a recipe, we generate a **Unified Feature String**. This single column concatenates the cleaned ingredients and processed instructions. By merging these fields, the engine simultaneously evaluates components and preparation methods to determine similarity.

5. Design Evolution and Deduplication A critical requirement for the SNACC pipeline is the removal of duplicate entries. Because recipe datasets often aggregate from multiple sources, identical recipes frequently appear with slight variations in formatting.

The strategy for handling this evolved from the **Original Design** to the **Current Implementation**:

- **Original Design (Integrated Pipeline):** The initial architecture (developed in [image_preprocessing_file.py](#)) was designed as a single-pass all-in-one script. This version performed text normalization, TF-IDF vectorization, and **internal deduplication** simultaneously, ensuring a unique "source of truth" before any data was exported.
- **Current Implementation (Modular Workflow):** For the final project build, the process was transitioned into a modular multi-stage workflow. While the primary [preprocess.py](#) handles formatting, a dedicated secondary utility ([remove_duplicates.py](#)) was implemented to perform a final integrity pass.
- **Implementation Note:** By separating the deduplication into its own run, we maintain a clear audit trail. We can compare the row counts before and after this extra step to verify how much redundant data was pruned from the final model.

6. Data Integrity Methods Within the current modular setup, deduplication is achieved through:

- **Exact Match Removal:** Dropping rows where the *Recipe Name* and *Ingredients* are identical.
- **Semantic Deduplication:** Flagging and removing recipes with identical **Unified Feature Strings**. This prevents the model from recommending redundant variations (e.g., multiple versions of "Classic Guacamole" that differ only by a pinch of salt), ensuring the user sees truly unique meal options.

7. Dataset Used

Link: <https://www.kaggle.com/datasets/wafaaelhusseini/extended-recipes-dataset-64k-dishes>

The dataset used in this project is an extended and feature-enriched version of the original Recipes Dataset: 64K Dishes (from Kaggle — a high-quality collection of 64,000+ structured recipes).

DATA PREPROCESSING ARCHITECTURE

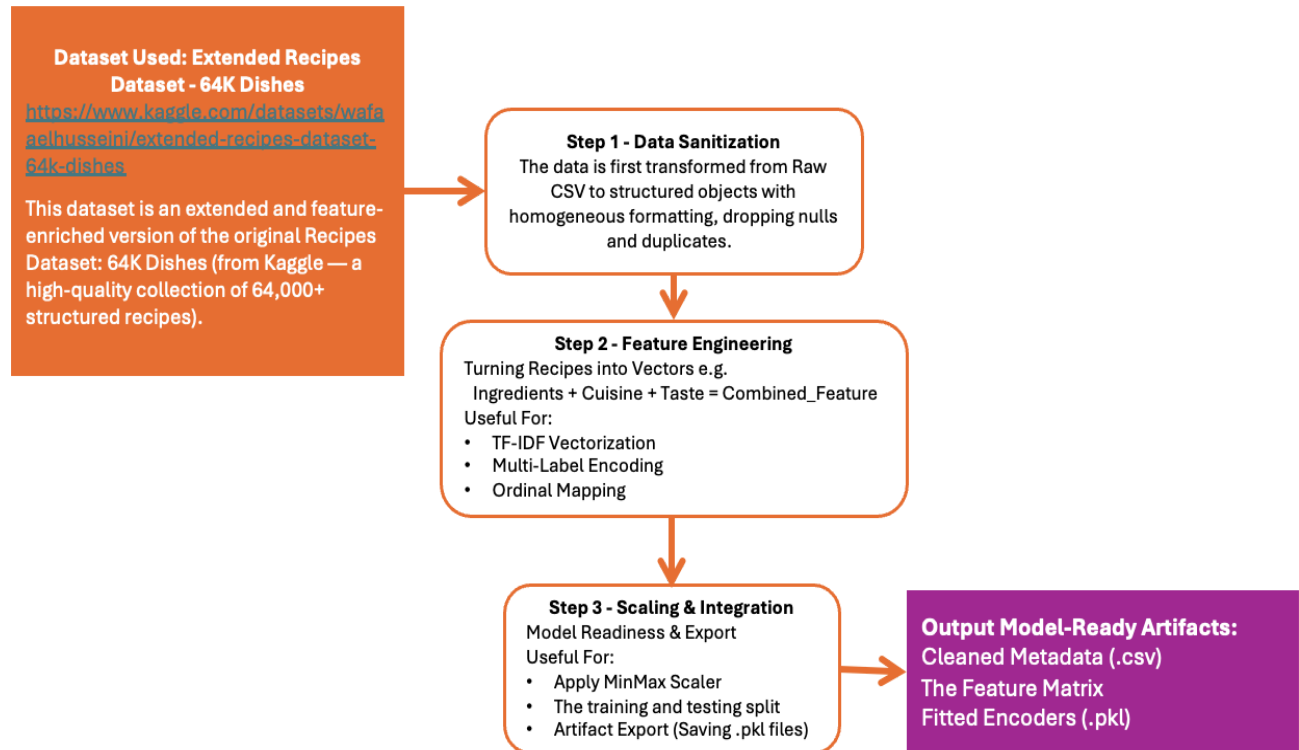


Fig 1. Data Preprocessing Basic Pipeline

C. Recommendation Engine: Model Pipeline and Filtering

The recommendation engine sits downstream of the preprocessing layer and is responsible for turning a user's pantry of ingredients plus any optional dietary and taste constraints into a ranked list of relevant recipes. The engine is designed around 2 principles:

1. Numerical features must reflect semantic similarity between an arbitrary user query and a curated recipe corpus
2. Hard constraints (dietary, taste) must be enforced before similarity ranking

1. Feature representation

Every cleaned recipe is represented by a single text field combining its cleaned ingredients and description. A TF-IDF vectorizer is fit over this corpus during training, capped at 5,000 terms with English stop words removed, so the model focuses on meaningful ingredient terms rather than common filler words. The fitted vectorizer is saved and reused at inference time, ensuring the user queries and recipes are compared in the same feature space.

2. Baseline similarity score

A Nearest Neighbors model using cosine distance is trained the full TF-IDF matrix and saved as a reference artifact. It is used during validation to confirm that similar recipes cluster together as expected.

3. Filter-Then-Rank Inference

At query time, the engine applies a filter then rank strategy:

1. Dietary filter: If the user selects a dietary preference (vegan, vegetarian, gluten-free, or dairy-free), only recipes flagged accordingly are kept
2. Taste filter: If one or more tastes are selected, the candidate set is further narrowed to recipes matching those tastes.
3. Ranking: The user's ingredients are vectorized with the saved TF-IDF vectorizer, cosine similarity is computed against the candidates, and the top-N most similar recipes are returned. The order guarantees that dietary and taste constraints are treated as hard requirements: a high-similarity recipe that violates a user's constraints can never surface to them.

4. Handling Empty Results

If filters eliminate every candidate, the engine returns an empty result rather than relaxing the user's constraints. The UI surfaces this clearly and invites the user to adjust their inputs, preserving the integrity of the dietary restrictions, which is critical for users with allergies or personal preferences.

5. Serving Layer

The engine is exposed through a FastAPI backend that returns recommendations as JSON, and a Streamlit frontend that calls the recommendation function directly for a responsive interactive experience.

6. Foodwaste Aware Ranking

On top of the baseline similarity score, the engine applies a re-ranking layer that aligns recommendations with the project's food-waste mission. Two signals are blended into the final score:

Pantry Coverage: Each candidate recipe is scored by the fraction of the user's input ingredients it actually consumes, so the system optimizes for "use what you have" rather than pure text similarity.

Perishability Prioritization: A shelf-life classifier sorts the user's ingredients into four tiers (highly perishable, perishable, moderate, pantry stable) and auto-flags items that should be used first. Recipes that consume those time-sensitive items receive an additional ranking boost, and the user can override the auto-detected list through a "Use First" control in the UI

The final score is a weighted blend of similarity, coverage, and perishability, and each recipe surfaces the missing ingredients the user would need to buy.

D. Architectural Diagrams

RECOMMENDATION ENGINE ARCHITECTURE

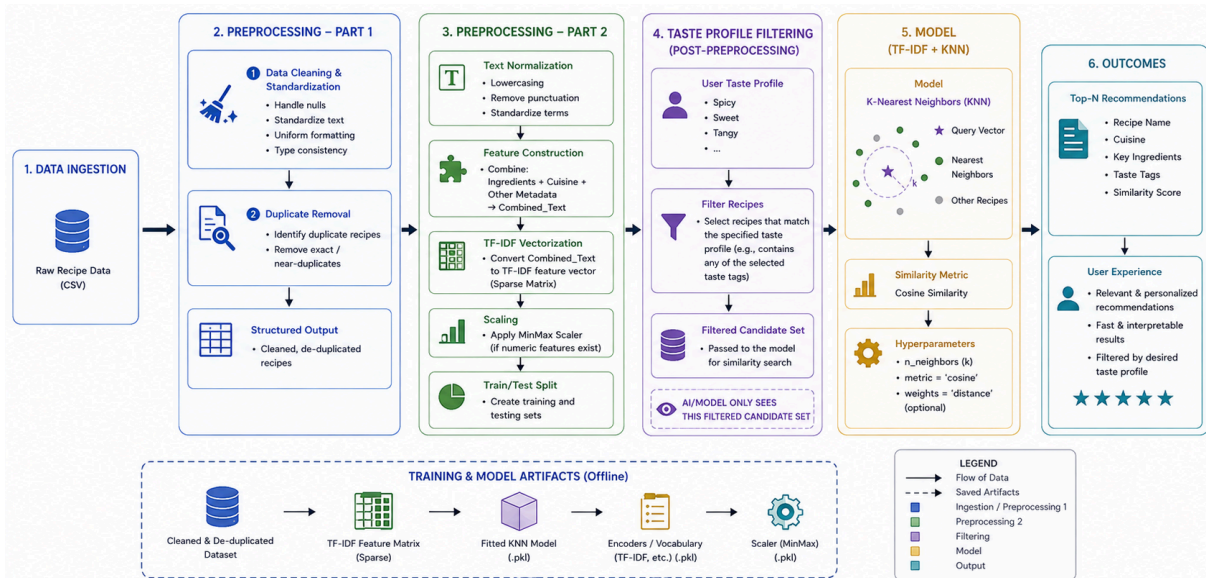


Fig 2. This directory structure reflects a modular and scalable architecture designed to decouple data preparation from real-time inference. The system is partitioned into three core layers: a data processing pipeline, a machine learning engine, and a user interface. The pipeline utilizes `image_preprocessing_file.py` and `remove_duplicates.py` as standalone utilities to transform raw CSV data into a high-fidelity, unique feature set. These cleaned datasets are then used by the model/ directory, where the TF-IDF and Nearest-Neighbors logic are trained and executed. Finally, the FastAPI backend serves as a bridge, allowing the Streamlit frontend to deliver interactive, low-latency recipe suggestions to the end user. This is version 1 of the diagram presented during the presentation.

Recipe Recommender

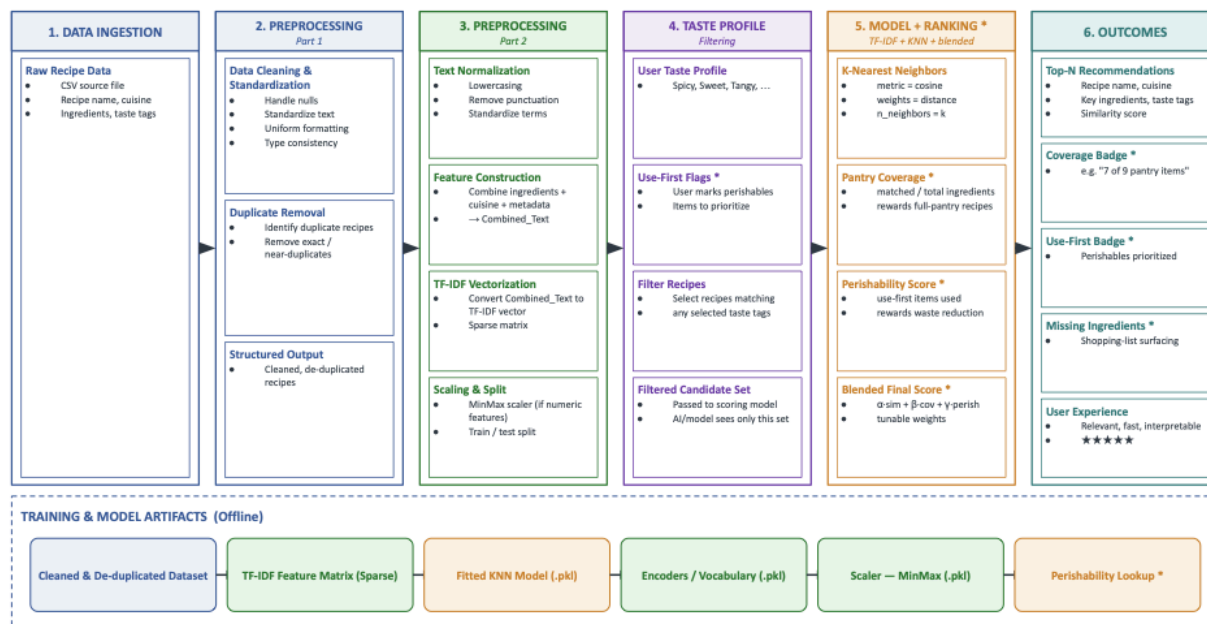


Fig 3. **Version 2** of the recommendation pipeline with a food-waste-aware re-ranking layer. After dietary and taste filters narrow the candidate set, three signals are computed in parallel: TF-IDF cosine similarity, pantry coverage (fraction of the user's input ingredients a recipe consumes), and perishability score (fraction of flagged "Use First" items a recipe uses). These blend into a single ranking score, and the output surfaces top-N recommendations with coverage and use-first badges plus a list of missing ingredients to buy.

E. Challenges and Solutions

1. Vocabulary Mismatch between Users queries and Recipe text

Challenge: A user typing “tomatoes” should match a recipe listing “tomato”; “freshly ground black pepper” and “pepper” should be treated as the same ingredient. Without normalization, TF-IDF would treat these as unrelated tokens and similarity scores would collapse.

Solution: The preprocessing pipeline applies lowercasing, regex-based punctuation stripping, English stop-word removal, and lemmatization before vectorization. Cooking-measurement tokens (“tbsp”, “cups”) are explicitly suppressed so they don't dilute the signal from actual ingredient terms.

2. Duplicate recipes inflating the result set

Challenge: The Kaggle dataset aggregates from multiple sources and frequently contains near-identical recipes. So “Classic Guacamole” might appear three times with only trivial wording differences. Without deduplication, all three would crowd out the other top results.

Solution: A two-tier deduplication strategy. Exact matches on (recipe_title, ingredients) are dropped first; then a semantic pass removes rows with identical combined_text strings. Splitting deduplication into its own utility (remove_duplicates.py) provides an audit trail where the row counts before and after are directly comparable.

3. Hard constraints vs. Soft ranking

Challenge: A naive nearest-neighbors approach can surface highly similar but constraint-violating recipes (e.g a chicken dish for a vegan user) because cosine similarity has no concept of dietary feasibility.

Solution: The filter-then-rank pipeline described above. Dietary and taste filters are applied to the dataframe before similarity is computed, making it impossible for an excluded recipe to ever appear in the ranking regardless of its TF-IDF proximity to the query.

4. Stale prebuilt Nearest Neighbors Index after Filtering

Challenge: The offline trained nearest neighbors index covers the full corpus. Once the user applies a dietary filter, the index's neighbor positions no longer correspond cleanly to the filtered dataframe, and the top-K results may collapse to fewer than K viable recipes.

Solution: Treat the saved NN model as a validation artifact rather than the inference path. At query time, re-vectorize only the filtered subset and compute the cosine similarity directly. This costs marginal compute on each request but guarantees correctness and exact top_n results when candidates exist.

5. Stringified Python Objects in CSV Columns

Challenge: Columns like `ingredients_canonical`, `cuisine_list`, and `tastes` are stored as string-encoded Python lists (`"['tomato', 'basil']"`) rather than native lists. Calling string methods directly on them produces garbage.

Solution: A safe literal wrapper around `ast.literal_eval` parses these fields into real lists, returning an empty list on malformed input rather than raising. So a single bad row never aborts the entire preprocessing pass.

6. Similarity-only ranking ignoring food-waste intent

Challenge: Pure cosine similarity can rank a recipe high even if it only uses one of the user's ingredients, defeating the project's food-waste mission. It also has no awareness of which ingredients are about to spoil.

Solution: A re-ranking layer blends similarity with two food-waste signals: pantry coverage (fraction of user ingredients consumed) and perishability coverage (fraction of flagged perishables consumed). A four-tier shelf-life lookup auto-detects expiring items, with a manual override so users can flag anything spoiling in their fridge. Recipes also surface the missing ingredients the user would need to buy.

F. Project Contribution Summary

Member	Role	Key Contributions & Technical Ownership
Faith Akinlade	Data Lead & Product Designer	- Product Strategy: Conceptualized the core SNACC application and designed the end-to-end user experience. - Data Engineering: Sourced the primary dataset and architected the reference feature engineering and preprocessing pipeline (normalization, lemmatization, and integrated deduplication). - Feature Implementation: Developed the taste-profile filtering logic and integrated these dynamic updates into the frontend UI.
Jared Simonetti	Systems & Research Lead	- Problem Definition: Conducted the literature review and expanded the project scope to address national food waste metrics. - Full-Stack Implementation: Developed the initial UI based on the design specifications and managed the data-to-model

		integration. - Integrity Utility: Authored the standalone deduplication logic to ensure final dataset quality.
Kalyani Chaudhary	Machine Learning Engineer	- Model Development: Developed the recommendation engine utilizing K-Nearest Neighbors (KNN) to calculate recipe similarity. - Mathematical Implementation: Leveraged the TF-IDF vectorized features and implemented Cosine Similarity metrics to determine feature-set proximity. - Model Validation: Assisted in testing recommendation accuracy and refining neighbor-selection parameters.
Ian Channer	Sustainability Feature Engineer	Coverage Ranking: Developed the pantry coverage scoring layer that blends ingredient utilization with cosine similarity, surfacing recipes that consume the largest share of the user's existing ingredients to directly address household food waste. Perishability Prioritization: Authored the shelf-life classification module and "Use First" ranking workflow that auto-detects perishable ingredients and re-weights recommendations to prioritize time-sensitive items. Quality Assurance: Authored the validation suite covering the coverage and perishability subsystems and integrated the missing-ingredients surfacing into the frontend UI.

G. Sources

Real World Application

[The Environmental Impact of Food Waste | Move For Hunger](#)

[Estimating the Cost of Food Waste to American Consumers](#)

[Reducing Wasted Food | Rescuing Leftover Cuisine](#)

[U.S. Population \(1950-2025\)](#)

[Food and Climate Change: Healthy diets for a healthier planet | United Nations](#)

<https://www.census.gov/>

Scikit-learn Documentation

<https://scikit-learn.org/stable/>

TF-IDF Vectorizer

https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

Nearest Neighbors

<https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.NearestNeighbors.html>

Tech

Kaggle Datasets – <https://www.kaggle.com>

Scikit-learn Documentation – <https://scikit-learn.org>

Streamlit Documentation – <https://docs.streamlit.io>

Python Official Site – <https://www.python.org>

GitHub – <https://github.com>

H. GitHub Repository

Public GitHub Repository: <https://github.com/FaithAkinlade/AI-Nutrition-Assistant>

Formatted IEEE Sources

- [1]
“Welcome to Python.org,” Python.org. Accessed: Apr. 19, 2026. [Online]. Available:
<https://www.python.org/>
- [2]
“U.S. Population (1950-2025).” Accessed: Apr. 19, 2026. [Online]. Available:
<https://www.macrotrends.net/global-metrics/countries/usa/united-states/population>
- [3]
“The Environmental Impact of Food Waste | Move For Hunger.” Accessed: Apr. 19, 2026.
[Online]. Available: <https://moveforhunger.org/the-environmental-impact-of-food-waste>
- [4]
“TfidfVectorizer,” scikit-learn. Accessed: Apr. 19, 2026. [Online]. Available:
https://scikit-learn/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
- [5]
“Streamlit Docs.” Accessed: Apr. 19, 2026. [Online]. Available: <https://docs.streamlit.io/>
- [6]
“scikit-learn: machine learning in Python — scikit-learn 1.8.0 documentation.” Accessed: Apr. 19,
2026. [Online]. Available: <http://scikit-learn.org/stable/>
- [7]
“scikit-learn: machine learning in Python — scikit-learn 1.8.0 documentation.” Accessed: Apr. 19,
2026. [Online]. Available: <https://scikit-learn.org/stable/>
- [8]
“NearestNeighbors,” scikit-learn. Accessed: Apr. 19, 2026. [Online]. Available:
<https://scikit-learn/stable/modules/generated/sklearn.neighbors.NearestNeighbors.html>
- [9]
“GitHub · Change is constant. GitHub keeps you ahead.,” GitHub. Accessed: Apr. 19, 2026.
[Online]. Available: <https://github.com/>
- [10]
U. Nations, “Food and Climate Change: Healthy diets for a healthier planet,” United Nations.
Accessed: Apr. 23, 2026. [Online]. Available:
<https://www.un.org/en/climatechange/science/climate-issues/food>

Formatted APA Sources

GitHub · Change is constant. GitHub keeps you ahead. (2026). GitHub. <https://github.com/>

Nations, U. (n.d.). *Food and Climate Change: Healthy diets for a healthier planet*. United Nations.

Retrieved April 23, 2026, from

<https://www.un.org/en/climatechange/science/climate-issues/food>

NearestNeighbors. (n.d.). Scikit-Learn. Retrieved April 19, 2026, from

<https://scikit-learn/stable/modules/generated/sklearn.neighbors.NearestNeighbors.html>

Scikit-learn: Machine learning in Python—Scikit-learn 1.8.0 documentation. (n.d.-a). Retrieved

April 19, 2026, from <http://scikit-learn.org/stable/>

Scikit-learn: Machine learning in Python—Scikit-learn 1.8.0 documentation. (n.d.-b). Retrieved

April 19, 2026, from <https://scikit-learn.org/stable/>

Streamlit Docs. (n.d.). Retrieved April 19, 2026, from <https://docs.streamlit.io/>

TfidfVectorizer. (n.d.). Scikit-Learn. Retrieved April 19, 2026, from

[https://scikit-learn/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.htm](https://scikit-learn/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html)

l

The Environmental Impact of Food Waste | Move For Hunger. (n.d.). Retrieved April 19, 2026, from

<https://moveforhunger.org/the-environmental-impact-of-food-waste>

U.S. Population (1950-2025). (n.d.). Retrieved April 19, 2026, from

<https://www.macrotrends.net/global-metrics/countries/usa/united-states/population>

Welcome to Python.org. (2026, April 14). Python.Org. <https://www.python.org/>